

COLLAGE OF COMPUTING AND INFORMATICS SCIENCE

COURSE TITLE: MICROPROCESSOR AND ASSEMBLY LANGUAGE

COURSE CODE: SEng2213

PROJECT TITLE:Traffic Light Controller using 8086 Microprocessor and 8255 PPI

Submitted by: **GROUP 1**

No	*GROUP NAME*	*ID*
1	BEWKET AFEWORK	MAU1600290
2	SENAIT KEDRU	MAU1601084
3	HAILEMARYAM EYASU	MAU1600639
4	KIDANU AMARE	MAU1600740
5	TADESSE TILAHUN	MAU1601128
6	YARED GIDEY	MAU1601295
7	MULUGETA LIUL	MAU1600957

Submitted to: **Mr.Tsehayehu.G**

Department : SOFTWARE ENGINNERING

MAU

YEAR : 3rd

Program: Bachelor's Degree

06/05/2018 E.C

TULU AWULIYA

ETHIOPIA

Traffic Light Controller using 8086 Microprocessor and 8255 PPI

Introduction

Traffic light control systems are a fundamental real-world application of embedded and microprocessor-based technologies, demonstrating how digital systems can manage physical environments. This project focuses on designing and implementing a functional traffic light controller using the Intel 8086 microprocessor and the 8255 Programmable Peripheral Interface (PPI). Since the 8086 lacks built-in input/output ports, the 8255 serves as an essential interface, enabling communication between the microprocessor and external hardware such as LED indicators.

The system is developed and tested within the Proteus simulation environment, providing a practical and educational approach to understanding microprocessor interfacing, assembly language programming, and real-time system control.

This project is particularly relevant for academic and laboratory settings, offering hands-on experience in hardware-software integration.

Objectives

The primary objectives of this project are:

1) To interface external devices, such as LEDs representing traffic signals, with the 8086 microprocessor.
2) To configure and utilize the 8255 PPI in Mode 0 for simple output operations.
3) To develop and implement an assembly language program that controls the traffic light sequence in a realistic manner.
4) To simulate the traffic light system in Proteus, ensuring correct timing and hardware interaction.
5) To gain practical knowledge in address decoding, timing delays, and system-level design using microprocessor-based architectures.

System Overview

The traffic light controller is built around the following core components:

- **8086 Microprocessor**: Acts as the central processing unit, executing the control program and managing data flow.
- **8255 PPI**: Provides 24 programmable I/O lines, with Port A configured to output signals to the traffic light LEDs.
- **74LS138 Decoder**: Performs address decoding to generate chip select signals, ensuring proper peripheral addressing.
- **LED Indicators**: Red, yellow, and green LEDs visually represent the traffic light states for two directions.
- **Current-Limiting Resistors**: 330Ω resistors protect the LEDs from excessive current.
- **Proteus Simulation Platform**: Used for designing, testing, and verifying the hardware and software integration.

The system operates by sending control signals from the 8086 to the 8255, which in turn drives the LEDs in a predefined sequence, simulating a typical traffic light cycle at an intersection.

Block Diagram Description

The Traffic Light Controller system is structured into several interconnected functional blocks, each performing a specific role in the overall operation. The diagram visually represents the flow of data, control signals, and addressing between the microprocessor and peripheral components.

Functional Blocks and Their Roles

1. 8086 Microprocessor

Role: Central control unit that executes the assembly language program stored in memory.

Functions:

- ✓ Sends data and control signals to the 8255 PPI via the system bus.
- ✓ Uses IN and OUT instructions for I/O operations.
- ✓ Generates timing and sequencing logic for traffic light states.

2. Address Latch (74LS373 / 74LS378)

Role: Separates multiplexed address and data lines (AD0–AD15) from the 8086.

Purpose: Latches the address during the first clock cycle, allowing the data bus to be used independently in subsequent cycles.

3. Address Decoder (74LS138)

Role: 3-to-8 line decoder that generates Chip Select (CS) signals for peripherals.

Function: Decodes higher-order address lines to enable the 8255 PPI only when a specific I/O address range is accessed, preventing bus conflicts.

4. 8255 Programmable Peripheral Interface (PPI)

Role: Acts as an I/O expansion chip, providing 24 programmable I/O pins.

Configuration:

- ✓ Operates in **Mode 0** (simple I/O).
- ✓ **Port A** (PA0–PA2) is configured as output and connected to the traffic light LEDs.
- ✓ **Control Register** is loaded with 80H to set Port A as output.

5. LED Indicator Circuit

Role: Visual representation of traffic signals.

Setup:

- ✓ PA0 → Red LED
- ✓ PA1 → Yellow LED
- ✓ PA2 → Green LED
- ✓ Each LED is connected in series with a 330Ω resistor to limit current.

6. Clock and Reset Circuitry

Role: Provides system clock and initialization signals.

Includes: Crystal oscillator for clock generation and a power-on reset circuit to initialize the 8086 and 8255 on startup.

Signal Flow and Interconnections

1. **Address/Data Bus**: Connects the 8086 to the latch, decoder, and 8255.
2. **Control Lines**:

- ✓ RD (Read) and WR (Write) control data direction.
- ✓ IO/M selects between I/O and memory operations.
- ✓ RESET initializes all components.

3. **Chip Select (CS)**: Generated by the 74LS138 to activate the 8255.
4. **Port Outputs**: PA0–PA2 drive the LEDs via current-limiting resistors.

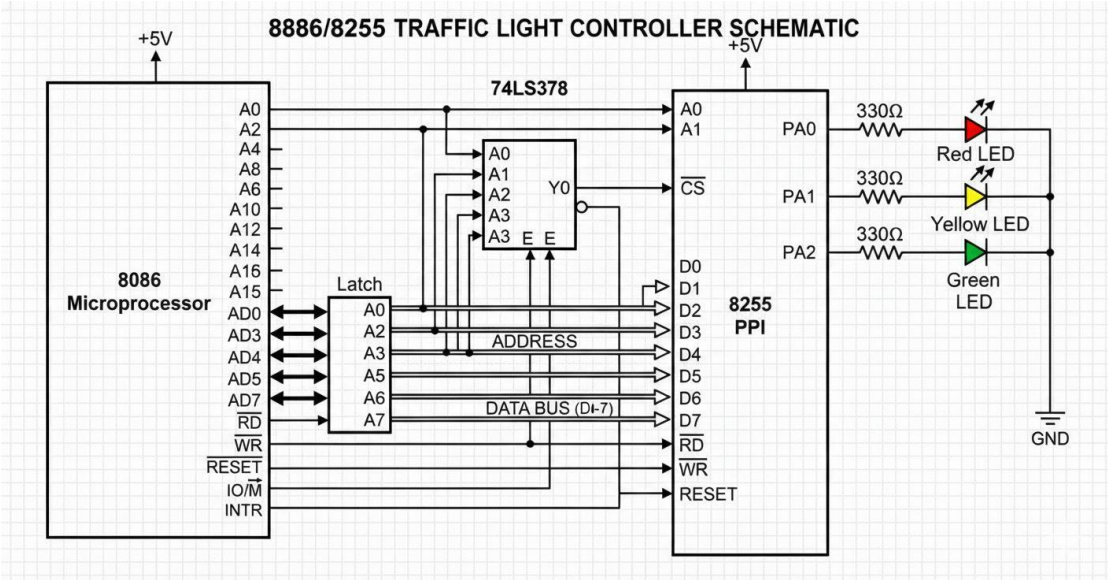


Figure 1: 8086–8255 Traffic Light Controller Schematic

Hardware Schematic

This schematic shows the complete hardware implementation of the Traffic Light Controller system. The **8086 microprocessor** is connected to the **8255 PPI** through the multiplexed address/data bus (AD0-AD15). The **74LS373 latch** separates address and data lines, while the **74LS138 decoder** generates the Chip Select (CS) signal for the 8255 based on address decoding. Port A of the 8255 (PA0-PA2) drives three LEDs (Red, Yellow, Green) through 330Ω current-limiting resistors. Control signals (RD, WR, IO/M, RESET) are properly connected to ensure correct timing and operation. Power supply (+5V) and ground connections complete the circuit.

Role of the 8086 Microprocessor

The 8086 microprocessor serves as the central processing unit with these key characteristics:

- **20-bit address bus** and **16-bit data bus**
- Uses **IN and OUT instructions** for I/O operations
- Requires **external hardware (8255)** for parallel I/O since it lacks built-in I/O ports

Key control signals used:

- **RD** – Read control signal
- **WR** – Write control signal
- **IO/M** – Selects between I/O and memory operations
- **RESET** – Initializes the system on power-up

8255 Programmable Peripheral Interface

Features:

- 24 programmable I/O lines
- Three 8-bit ports: Port A, Port B, Port C
- Supports three modes: Mode 0, Mode 1, and Mode 2

Configuration Used:

- ✓ **Mode 0** – Simple output mode (no handshaking)
- ✓ **Port A** configured as output port
- ✓ **Control Word**: 80H (10000000 in binary)

Control Word Format:

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	0	0	0	0	0

Where: D7=1 (Mode Set flag), D6D5=00 (Port A Mode 0), D4=0 (Port A Output), D3=0 (Port C Upper Output), D2=0 (Mode selection for Group B), D1=0 (Port B Output), D0=0 (Port C Lower Output)

Port Assignment and LED Connections

Table 1: Port Assignment and LED Connections

LED Color	8255 Port	Pin Assignment	Connection Details
Red	Port A	PA0	Connected through 330Ω resistor to +5V
Yellow	Port A	PA1	Connected through 330Ω resistor to +5V
Green	Port A	PA2	Connected through 330Ω resistor to +5V

Note: Each LED is connected in **active-high configuration** – when the port pin goes HIGH, the LED turns ON.

Address Decoding Using 74LS138

The **74LS138 3-to-8 line decoder** performs critical address decoding:

- **Inputs**: Three address lines (A15-A13 or similar) from the microprocessor
- **Outputs**: Eight active-low chip select signals
- **Function**: Generates the Chip Select (CS) signal for the 8255 when specific address lines match the decoding logic
- **Purpose**: Ensures the 8255 responds only to its designated I/O address range, preventing conflicts with other peripherals

Typical Connection:

- Address lines A2-A0 connect to the 8255 for port selection (Port A, B, C, Control)
- Higher address lines (A15-A3) connect to the 74LS138 for chip selection

Software Design Overview

The software is written in 8086 assembly language and performs these sequential tasks:

1. **Initialization**: Set all traffic lights to RED state
2. **Pattern Loading**: Load predefined traffic light patterns from memory
3. **Output Control**: Send data to Port A of 8255 using OUT instruction
4. **Timing Control**: Generate precise delays using BIOS interrupt
5. **Sequence Looping**: Continuously cycle through traffic patterns

Key Software Components:

- **OUT instruction**: Sends data to I/O ports
- **SI register**: Acts as pointer to traffic pattern table
- **INT 15H interrupt**: Provides accurate timing delays
- **Memory table**: Stores all traffic light states

Assembly Language Program (Source Code)

```
; controlling external device with 8086 microprocessor
; realistic test for c:\emu8086\devices\Traffic_Lights.exe
#start=Traffic_Lights.exe#
name "traffic"
mov ax, all_red
out 4, ax
mov si, offset situation
next:
mov ax, [si]
out 4, ax
; wait 5 seconds (5 million microseconds)
mov cx, 4Ch
mov dx, 4B40h
mov ah, 80h
int 15h
add si, 2
cmp si, sit_end
jb next
```

```
mov si, offset situation
jmp next
situation dw 0000_0011_0000_1100b
s1 dw 0000_0110_1001_1010b
s2 dw 0000_1000_0110_0001b
s3 dw 0000_1000_0110_0001b
s4 dw 0000_0100_1101_0011b
sit_end = $
all_red equ 0000_0010_0100_1001b
```

Program output

This section contains **simulation screenshots** captured from the Proteus simulation environment, demonstrating the sequential operation of the traffic light controller as programmed in the 8086 assembly language.

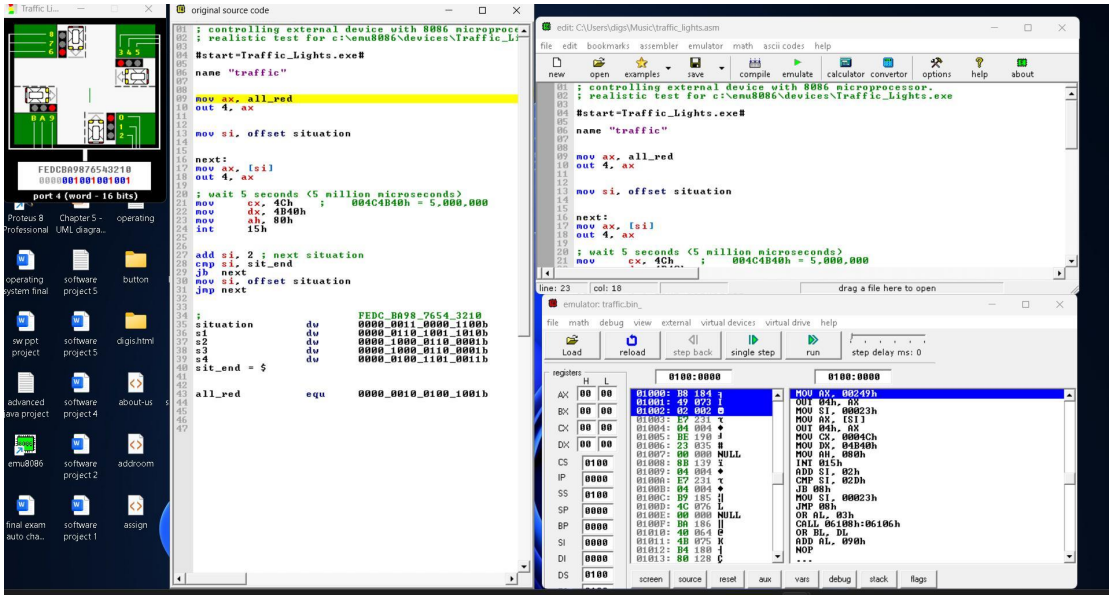


Fig 2: Initial State (All Red)

Visual Content:

- ✓ Shows the **Proteus simulation workspace** with all components visible.

- ✓ **8086 microprocessor**, **8255 PPI**, **74LS138 decoder**, and **LED arrays** are connected as per the schematic.
- ✓ **All six LEDs** (3 for each traffic direction) are illuminated in **RED**.
- ✓ This represents the **system initialization state** where all traffic is stopped for safety.
- ✓ The assembly code has just executed `mov ax, all_red` and `out 4, ax`.

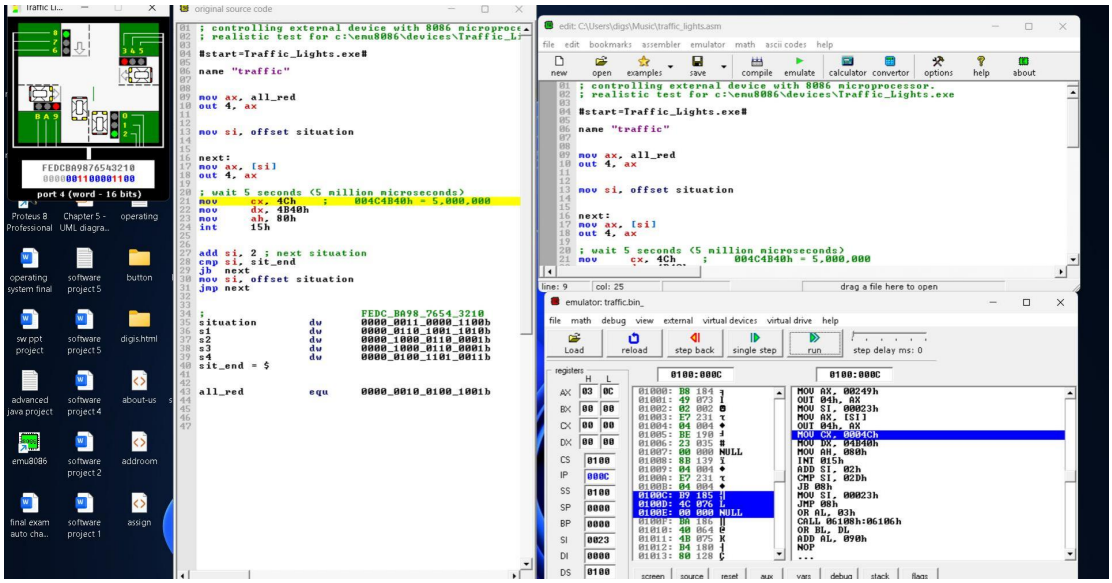


Fig 3: Direction 1: Green Active

Visual Content:

- ✓ **First direction** (left-side LEDs): **GREEN LED is ON**, Red and Yellow are OFF.
- ✓ **Opposite direction** (right-side LEDs): **RED LED is ON**, Green and Yellow are OFF.
- ✓ This corresponds to the traffic state where **one road has right of way** while cross traffic is stopped.
- ✓ Represents the output of the first traffic pattern from memory (likely `situation` or `s1` from the code).
- ✓ Current flow indicators show active signals on PA2 (Green) for one direction.

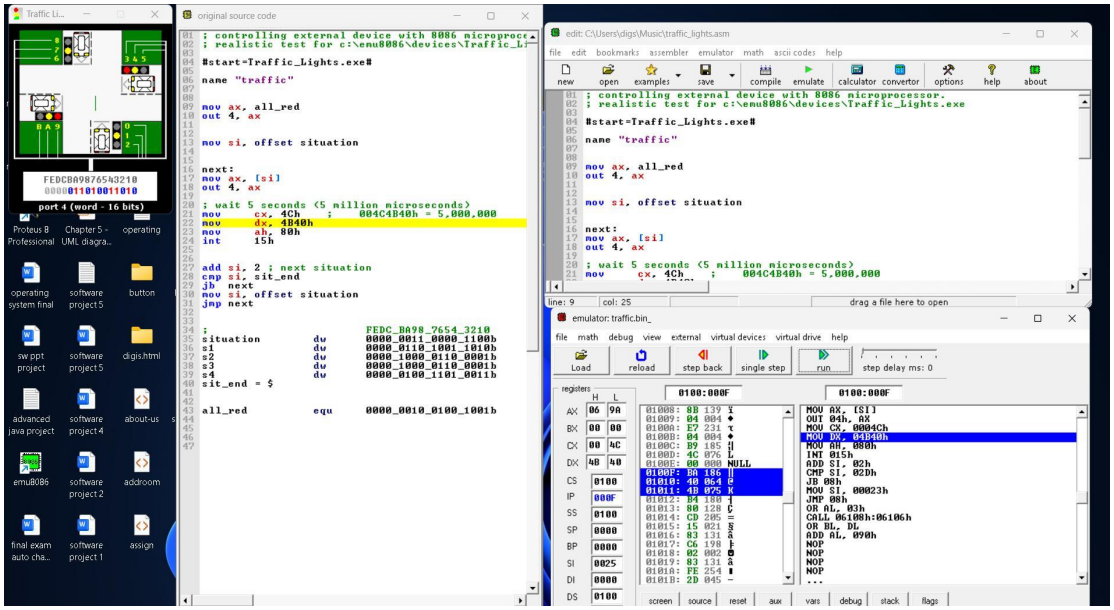


Fig 4:Direction 1: Yellow Transition

Visual Content:

- ✓ **First direction**: **YELLOW LED is ON** (transitional phase), Green is OFF.
- ✓ **Opposite direction**: Still shows **RED** (waiting).
- ✓ This is the **"prepare to stop"** phase, occurring after the green light and before red.
- ✓ The 5-second delay has elapsed, and the program has advanced to the next state in the sequence.
- ✓ Shows proper timing implementation between state transitions.

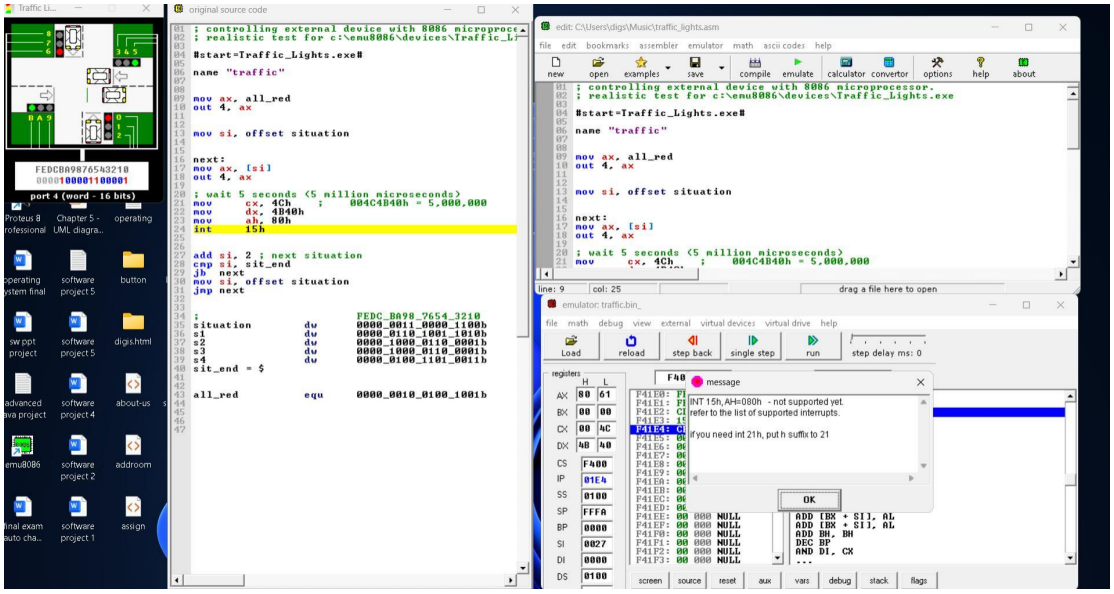


Fig 5:Direction 2: Green Active

Visual Content:

- ✓ **Traffic flow has switched directions.**
- ✓ **First direction**: Now shows **RED**.
- ✓ **Opposite direction**: **GREEN LED is ON**, Red and Yellow are OFF.
- ✓ This demonstrates the **alternating traffic control** where right-of-way transfers between roads.
- ✓ Corresponds to states s2 or s3 from the assembly code memory table.

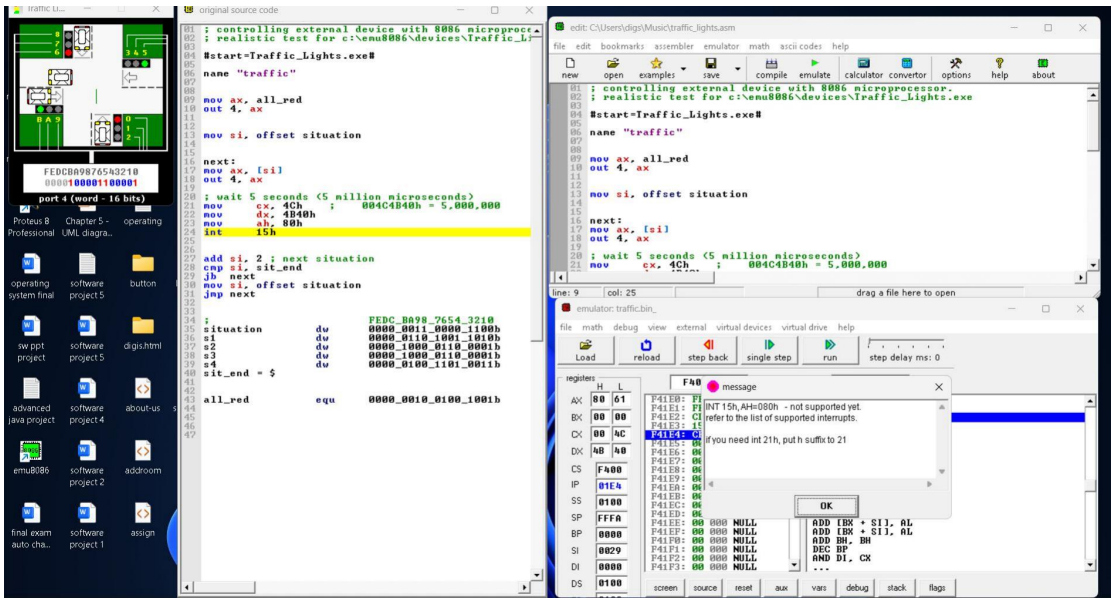


Fig 6:Direction 2: Yellow Transition

Visual Content:

- ✓ **Opposite direction**: **YELLOW LED is ON**, Green is OFF.
- ✓ **First direction**: Still shows **RED**.
- ✓ This completes the **second transitional phase** before the sequence returns to the beginning.
- ✓ The system is about to cycle back to the initial all-red state or continue to the next pattern.
- ✓ Shows the **completion of one full traffic light cycle**.

Explanation of Program Logic

This section explains how the assembly language code controls the traffic light system:

1)OUT 4, AX Instruction

- ➢ Sends the 16-bit value in the AX register to I/O port address 4
- ➢ Port address 4 corresponds to Port A of the 8255 PPI
- ➢ Each bit pattern in AX controls specific LEDs (bits 0-2 for one direction, bits 8-10 for the other)

2)SI Register as Pointer

- ➢ SI (Source Index) register points to memory locations containing traffic states
- ➢ Initialized with `mov si, offset situation` to point to the first traffic pattern
- ➢ Incremented by 2 (`add si, 2`) to move to the next 16-bit pattern
- ➢ Loops back to beginning when reaching `sit_end`

3)Traffic State Words

- ➢ Each `dw` (define word) represents a complete traffic configuration
- ➢ Example: `0000_0011_0000_1100b` controls 6 LEDs (3 per direction)
- ➢ Bit positions correspond to specific LEDs connected to Port A
- ➢ Multiple patterns create the traffic sequence cycle

4)BIOS Interrupt INT 15H

- Function 86H (Wait) provides precise timing delays
- CX:DX registers contain delay in microseconds
- mov cx, 4Ch and mov dx, 4B40h sets 5,000,000 microseconds (5 seconds)
- Preserves system timing accuracy during simulation

Timing and Delay Calculation

Detailed Calculation:

- 5 seconds delay between traffic state changes
- 1 second = 1,000,000 microseconds
- 5 seconds = 5 × 1,000,000 = 5,000,000 microseconds
- Hexadecimal conversion: $5{,}000{,}000_{10} = 004C4B40_{16}$
- Loaded as: CX = $004C_{(16)}$, DX = $4B40_{(16)}$

Why This Timing:

- Matches real-world traffic light intervals
- Provides visible observation time in simulation
- Demonstrates accurate timekeeping in assembly programming

Simulation in Proteus

1. **Component Placement**: Add 8086, 8255, LEDs, decoder
2. **Program Loading**: Load compiled assembly HEX file
3. **Connections**: Wire buses, control signals, power
4. **Run**: Start simulation and verify initialization
5. **Observe**: Monitor 5-second LED sequence changes

Applications

- Traffic junction control systems
- Embedded systems education
- Microprocessor laboratory experiments

- ✧ Industrial automation basics

Advantages

- ✧ Simple, modular design
- ✧ Easily expandable
- ✧ Demonstrates real-world I/O control

Limitations

- ✧ Fixed timing (non-adaptive)
- ✧ No sensor-based control
- ✧ Requires manual programming changes

Conclusion

This project successfully demonstrates 8086 microprocessor control of external devices using the 8255 PPI, simulating real-world traffic light operation through hardware interfacing and assembly programming. It serves as a complete educational tool suitable for academic evaluation and practical learning in embedded systems.

References

- ➢ **emu8086 Guide** – Used for writing and testing assembly code.
- ➢ **8086 Manual** – Provided microprocessor specs and control signal details.
- **8255 Datasheet** – Guided port configuration and Mode 0 setup.
- ➢ **Proteus Docs** – Supported circuit simulation and result validation.